EXPLOITS UNIVERSITY

BSc INFORMATION COMMUNICATION AND TECHNOLOGY

BICT 3202 · OBJECT-ORIENTED ANALYSIS AND DESIGN

WEEK 6

Object-Oriented Analysis — Object Modelling II

PROGRAMME	BSc Information Communication and Technology
COURSE CODE / YEAR	BICT 3202 · Year 3
LECTURER	Francis Fweta — ICT Lecturer
DELIVERY	2h Lecture · 1h Tutorial · 1h Practical · 10h Independent Learning

Prepared by Francis Fweta · Exploits University · Week 6 of 16

Contents

1. Week 6 Introduction

Week 5 taught students to identify and model classes, objects, attributes, operations, links, associations, multiplicity, generalization and specialization. Week 6 builds on those concepts. The progression is: **Week 5** — what objects/classes exist, how they are related, and generalization/specialization; **Week 6** — how objects form whole-part structures (aggregation), how characteristics are inherited (inheritance), how different objects respond differently (polymorphism), and how related classes are organized (grouping).

Aggregation, inheritance and polymorphism are not isolated programming concepts. In OOAD they help us construct models that represent real systems in a reusable and understandable way. The formal UML specification defines aggregation as a property of an association end and distinguishes shared aggregation from composite aggregation.

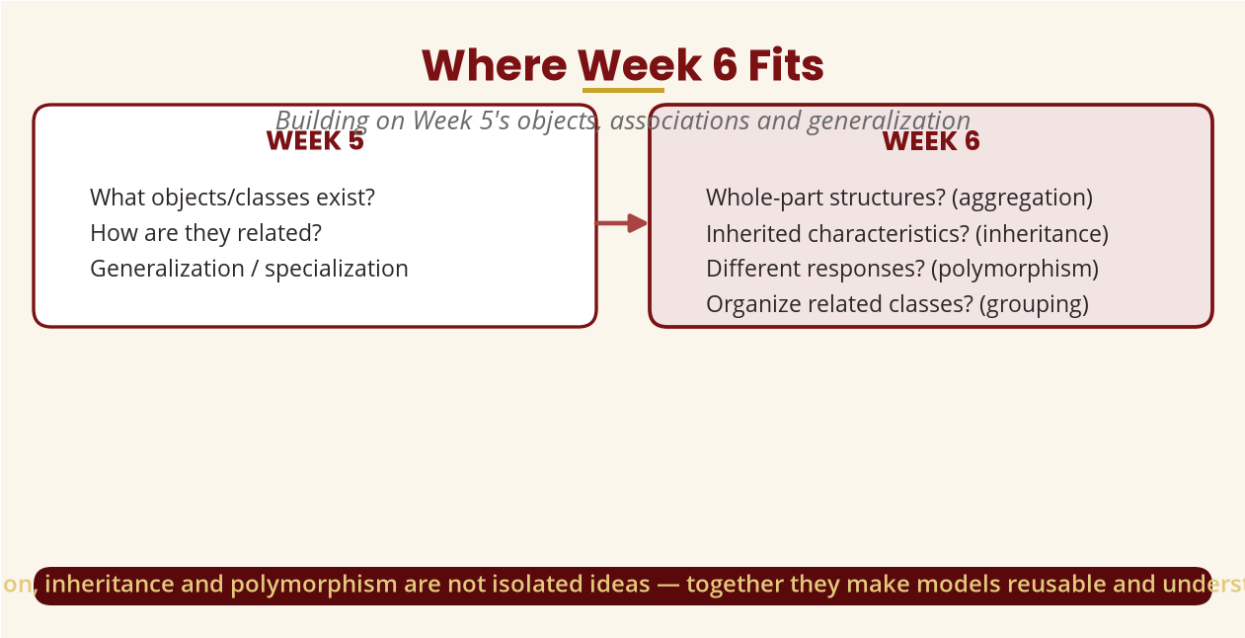

Figure 1 — Where Week 6 fits in the module

2. Learning Outcomes

By the end of this week, a student should be able to:

- Define aggregation and explain the whole-part relationship.
- Distinguish ordinary association, aggregation and composition.
- Explain shared aggregation and composite aggregation.
- Represent aggregation using UML notation (hollow vs filled diamond).
- Explain inheritance and its relationship to UML generalization.
- Identify superclass and subclass, and inherited attributes and operations.
- Distinguish inheritance (is-a) from association and aggregation (has-a).
- Explain polymorphism and method overriding with real-world examples.
- Explain why polymorphism supports extensibility.
- Explain grouping/classification and organize classes into packages.

- Develop a complete class hierarchy and evaluate modelling choices.

PART A — AGGREGATION

3. What Is Aggregation?

What does it mean when one object is made up of, or contains, other objects? Consider a university: a university has departments, and a department has lecturers, students and courses. We can represent `University ◇——— Department` — a **whole-part relationship**: the university is the whole, the department is a part. In simple English: **aggregation is a relationship in which one object represents a whole or collection that is related to other objects representing its parts** — a Library contains Books; a Department has Lecturers.

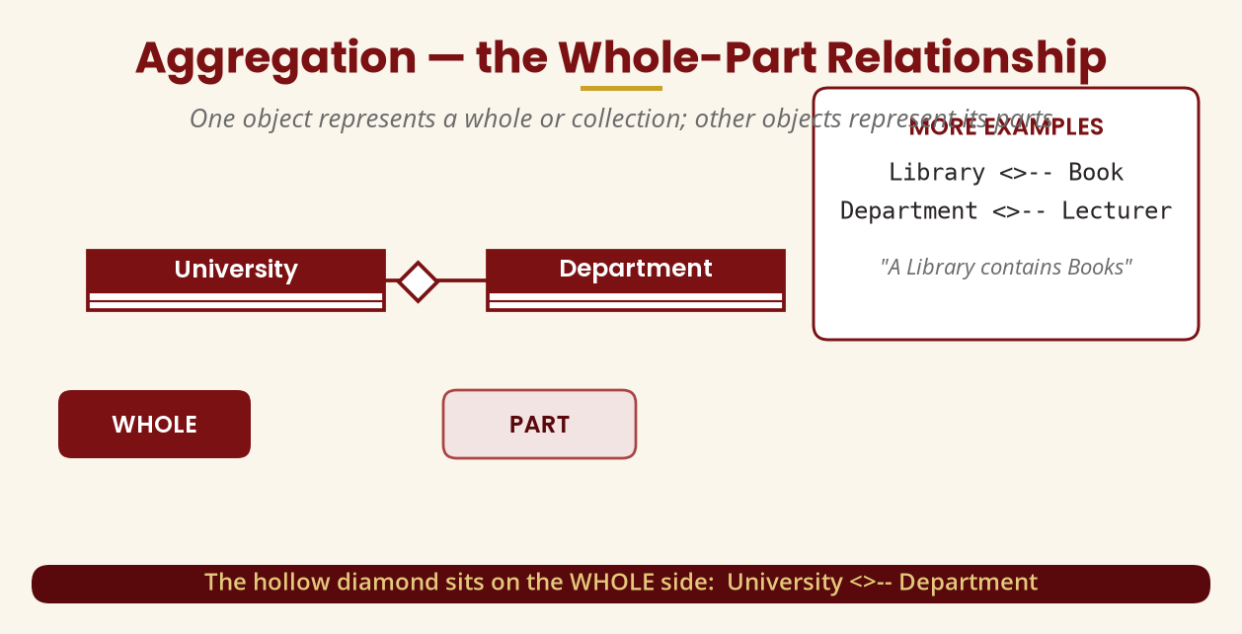

Figure 2 — The whole-part relationship and the hollow diamond

4. The UML Aggregation Symbol

Aggregation is represented with a **hollow diamond**, placed on the **whole** side. This is different from the **filled diamond** used for composition. The distinction between aggregation and ordinary association is extremely important: an association is a general relationship ("a Student is related to a Course"), while aggregation is a whole-part relationship ("a Department is an aggregate involving Lecturers").

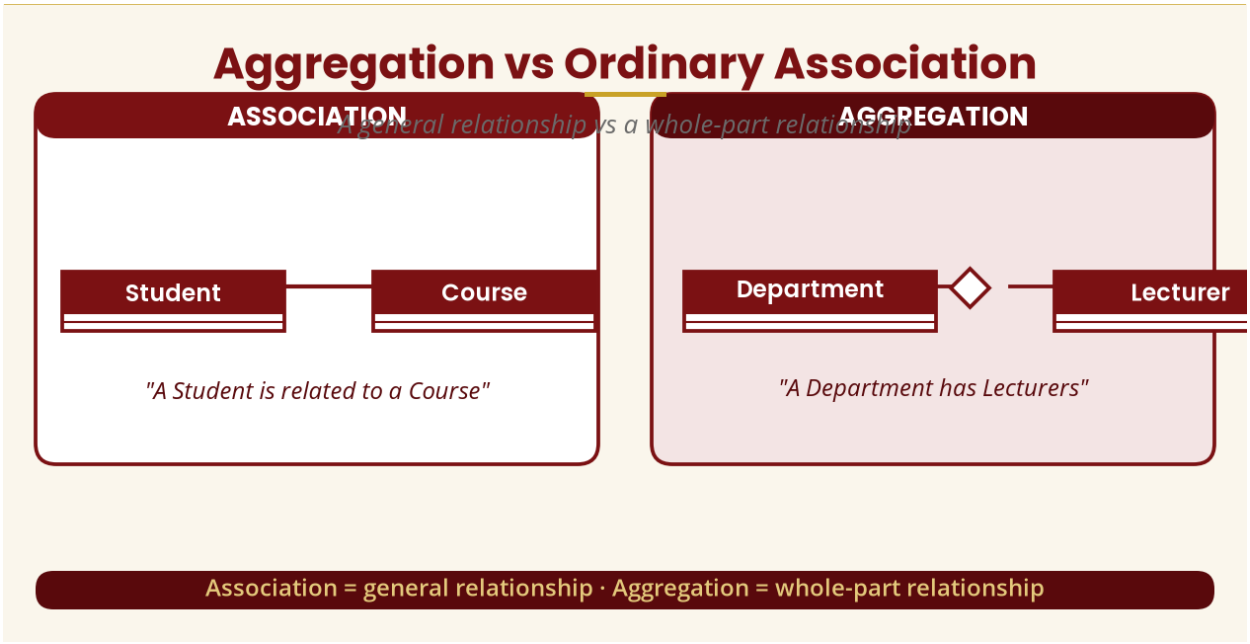

Figure 3 — Aggregation vs ordinary association

5. Aggregation Does NOT Mean "the Part Dies with the Whole"

This is the easiest way to distinguish aggregation from composition. If a Department is reorganized or removed, does the Lecturer cease to exist? No — the lecturer can be assigned to another department. A football team consists of players, but if a player leaves the club the player does not cease to exist; if a classroom is closed, the students still exist. These are examples of **weak/shared** whole-part aggregation, where the parts have an independent existence.

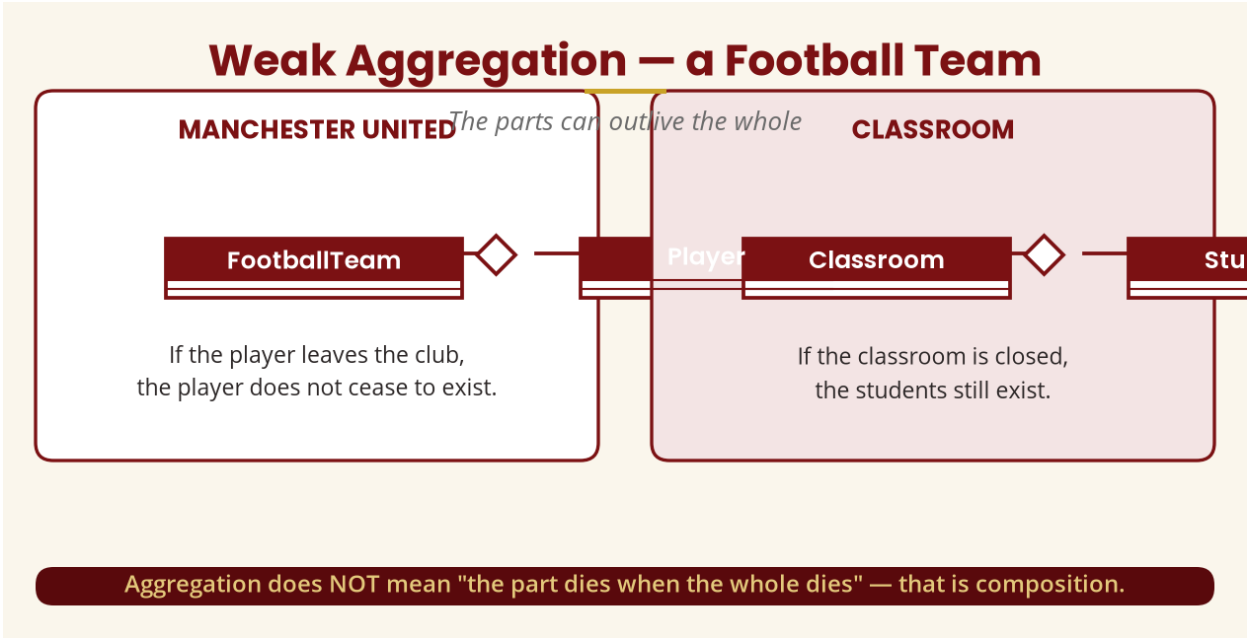

Figure 4 — Weak aggregation: parts that outlive the whole

6. Composition — the Strong Form

Composition is a stronger form of whole-part relationship, drawn with a **filled/black diamond** (`House ◆——— Room`). UML identifies composite aggregation as the form where the composite object has responsibility for the existence and storage of the composed parts; a part can be included in at most one composite at a time. If an order is deleted, its order lines normally have no independent meaning — so `Order ◆——— OrderLine` is a much stronger relationship than `Team ◇——— Player`. The relationship therefore tells us something about the **lifecycle and ownership** of objects.

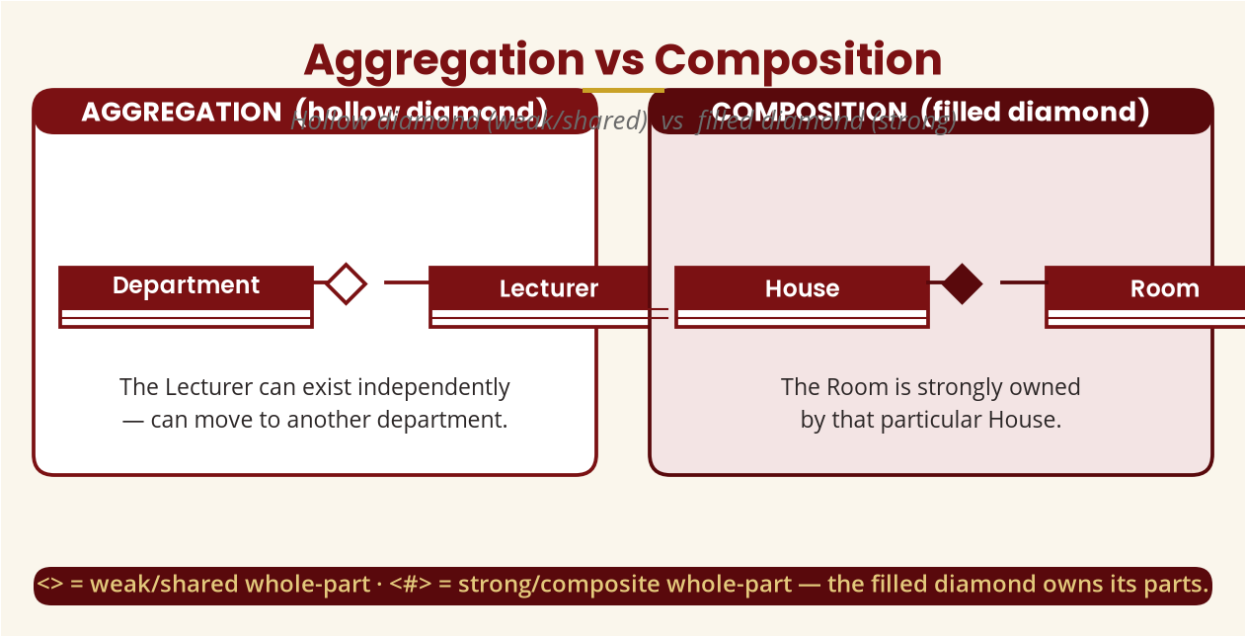

Figure 5 — Aggregation (hollow) vs composition (filled)

7. Aggregation with Multiplicity — and a Warning

Aggregation can also carry multiplicity: `University 1 ◇——— 1..* Department` reads "one university has one or more departments". But do not assume "contains" automatically means aggregation — "a customer has an address" does not automatically mean `Customer ◇——— Address`. The analyst must understand the domain and lifecycle. **The goal is not to use every possible UML symbol, but to choose the relationship that best represents the requirements.**

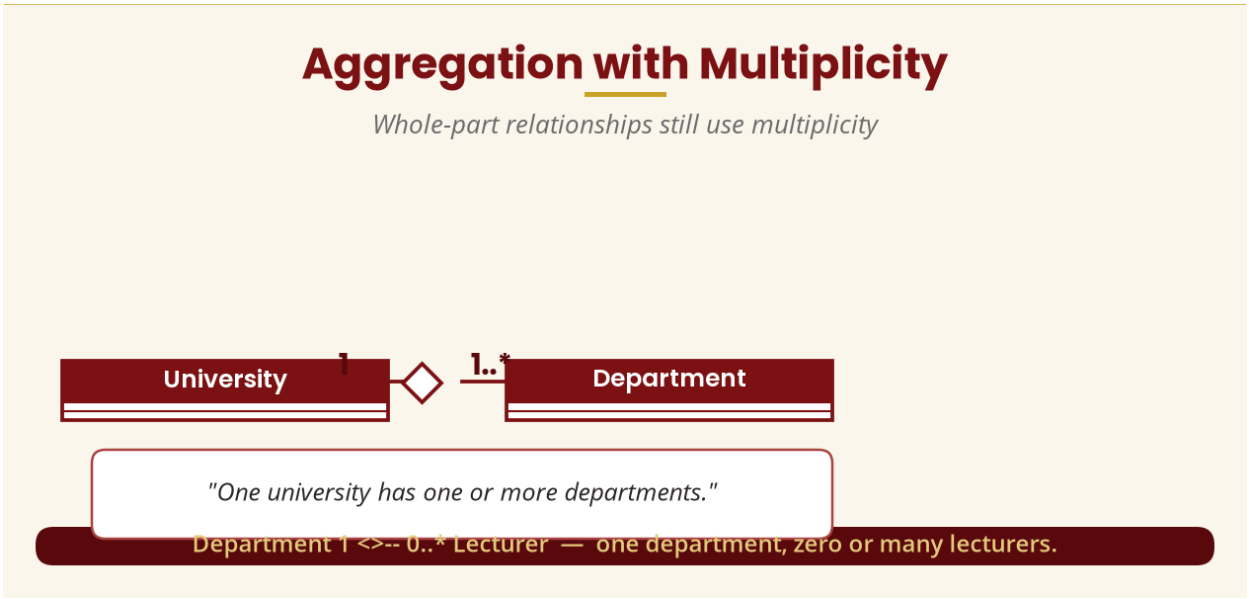

Figure 6 — Aggregation with multiplicity

PART B — INHERITANCE

8. What Is Inheritance?

Inheritance is a mechanism through which a more specific class can acquire characteristics and behaviour from a more general class. A Car inherits the characteristics of Vehicle, and a Motorcycle also inherits them. In UML this relationship is represented through **generalization**; the UML specification states that an instance of the specific classifier is also an instance of the general classifier, and that the specific classifier inherits features of the general classifier.

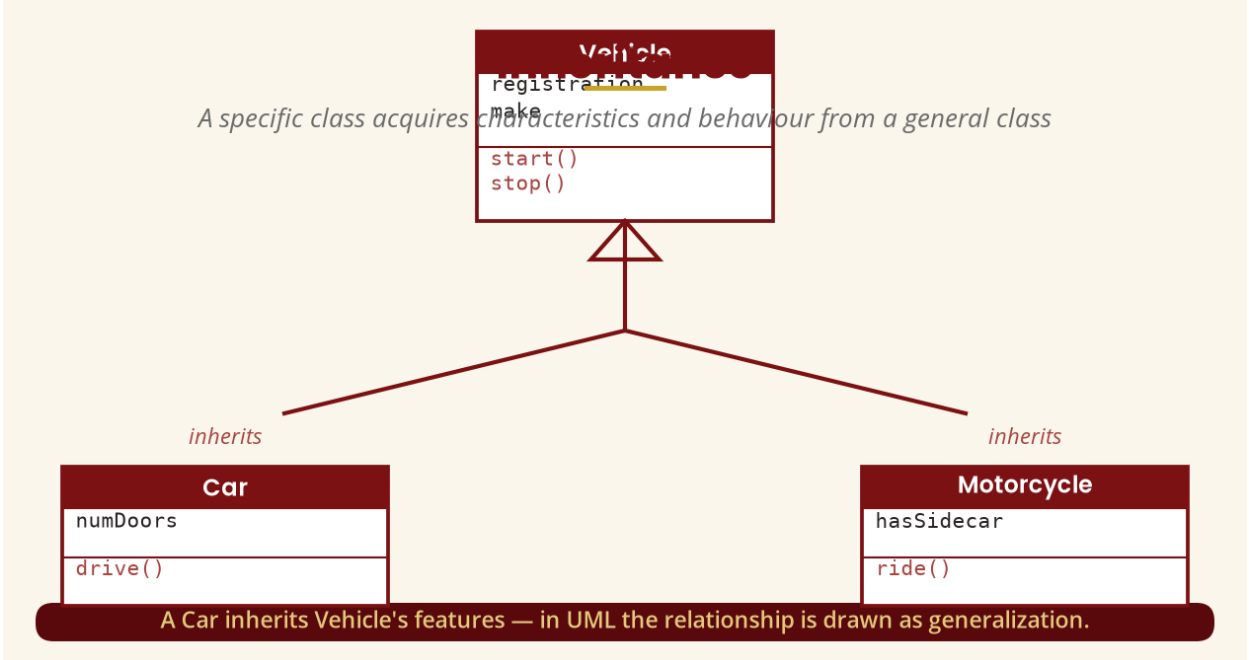

Figure 7 — Inheritance drawn as UML generalization

9. Generalization vs Inheritance

Students often ask whether generalization and inheritance are the same. They are closely related, but we should be precise: **UML terminology** calls the relationship in the model **generalization**; **object-oriented programming terminology** calls the implementation mechanism **inheritance**. A UML generalization relationship can be implemented using inheritance in an object-oriented programming language.

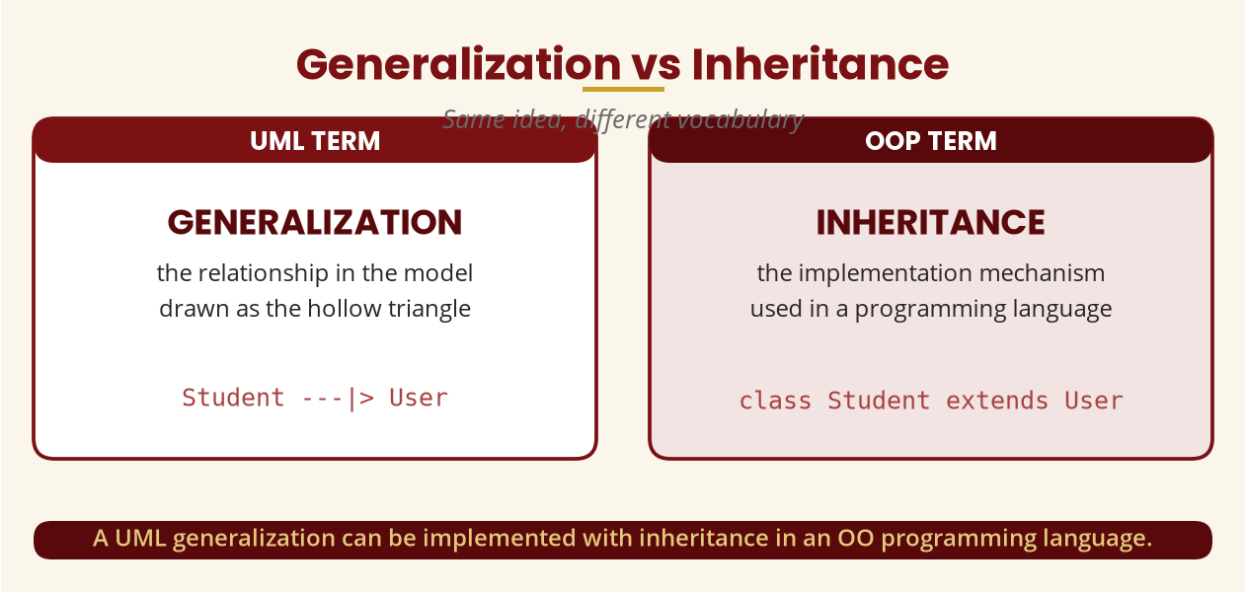

Figure 8 — Generalization (UML) vs inheritance (OOP)

10. What Does the Child Inherit?

Suppose Employee has employeeID, name, email, login() and logout(). A Manager with department and approveLeave() can conceptually have employeeID, name, email, department, login(), logout() and approveLeave() — without re-declaring the common features. Inheritance supports **reuse** (common characteristics defined once), **consistency** (shared behaviour managed in one class), **specialization** (subclasses add their own features), **polymorphism** (substitutability) and **maintainability** (changes managed centrally).

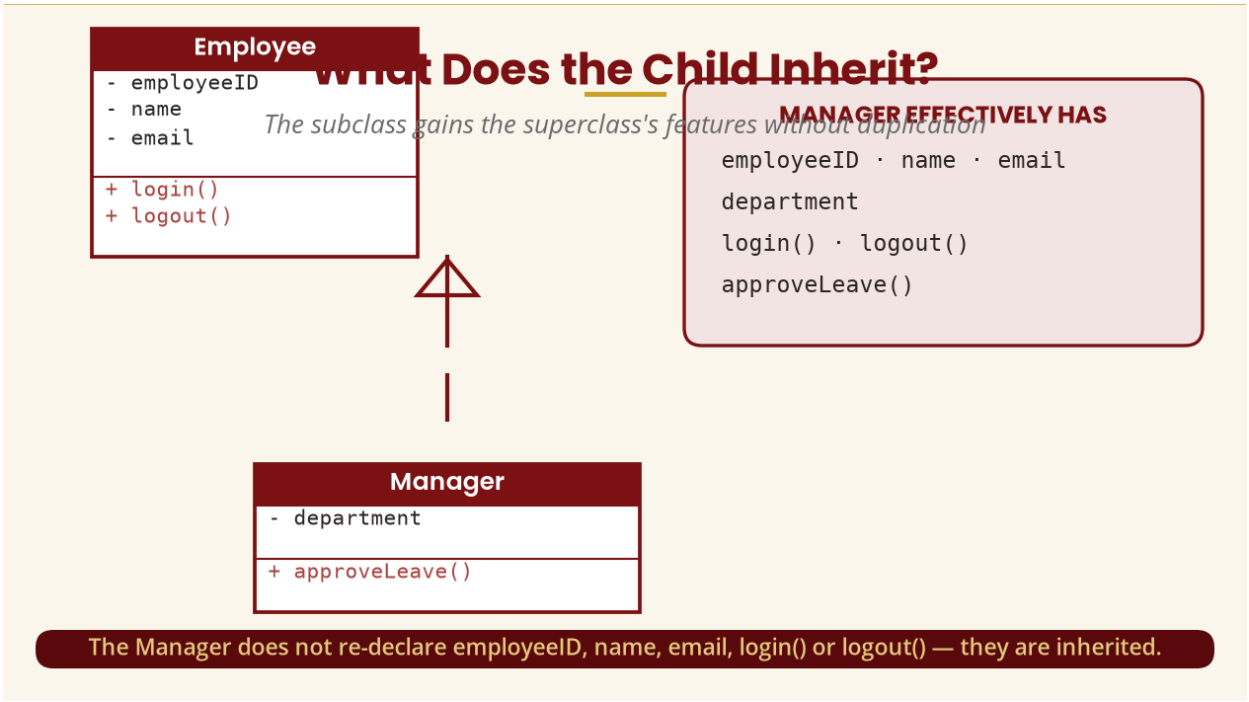

Figure 9 — What a subclass inherits from its superclass

11. The Is-A Rule — and Inheritance vs Aggregation

Inheritance should generally make sense as an **is-a** relationship: a Dog is an Animal (good), a Car is a Vehicle (good), but a Car is an Engine (no) and a Student is a Course (no). This is one of the most common examination questions: **inheritance represents IS-A** (Car --|> Vehicle), while **aggregation represents HAS-A / PART-OF** (Car o-- Engine).

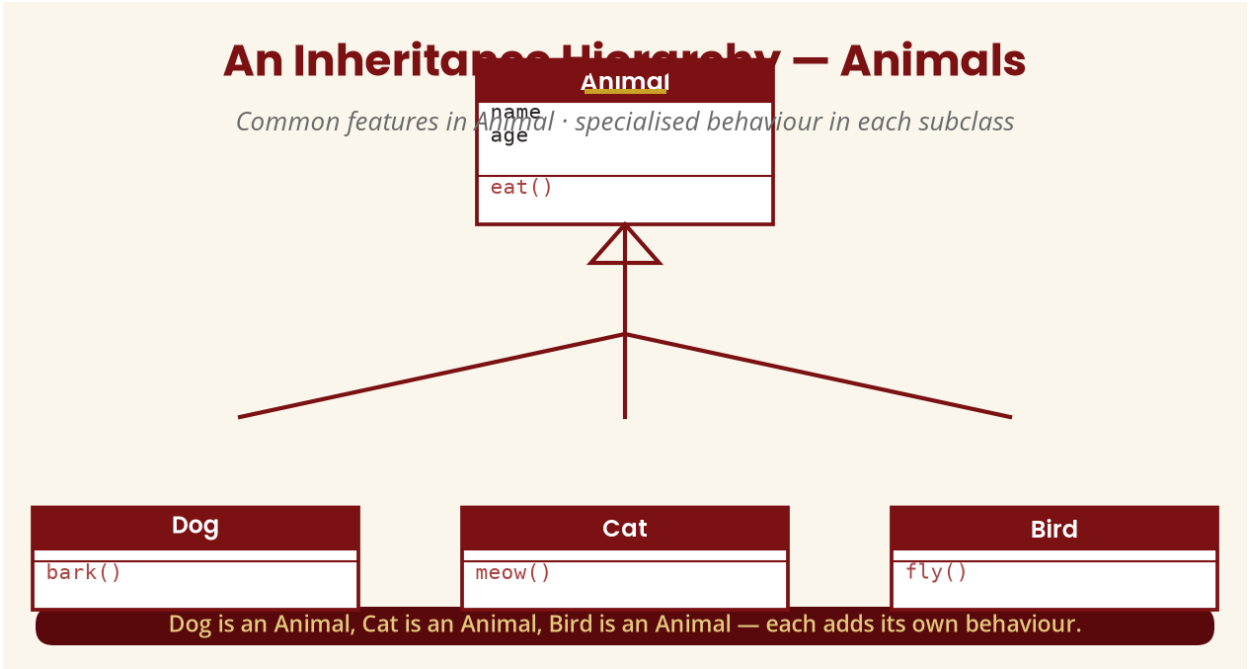

Figure 10 — An animal hierarchy with specialised behaviour

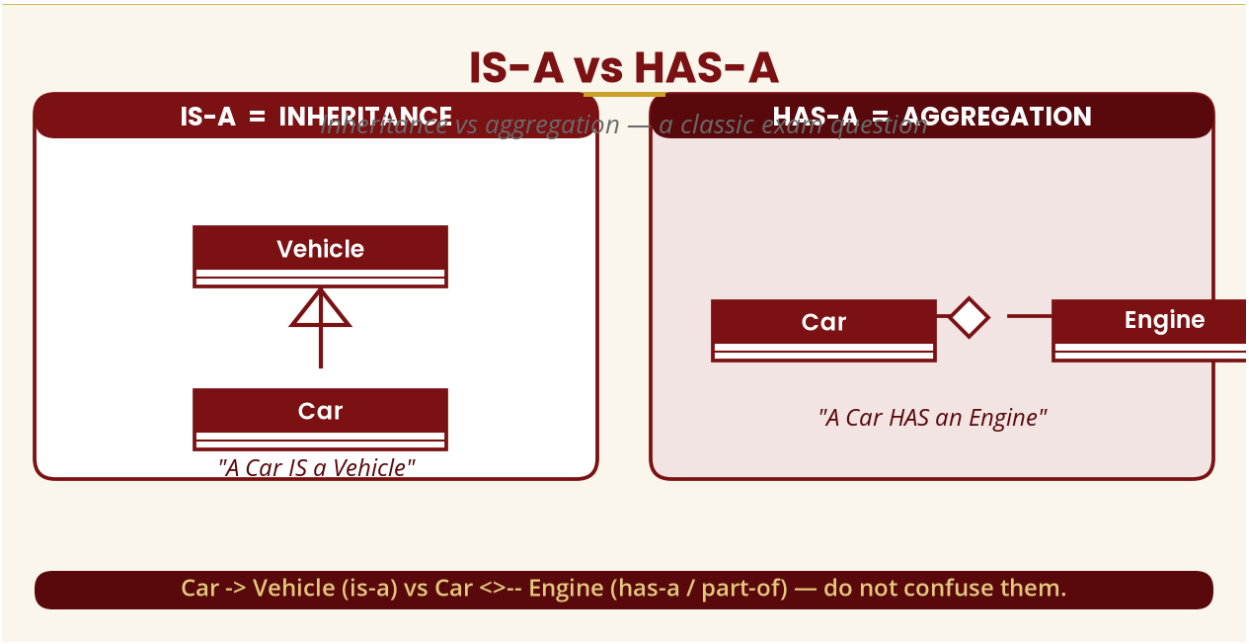

Figure 11 — IS-A (inheritance) vs HAS-A (aggregation)

PART C — POLYMORPHISM

12. What Is Polymorphism?

The word polymorphism comes from Greek roots meaning roughly **“many forms”**. In object-oriented systems, polymorphism allows the same operation/message to be associated with different implementations or behaviours depending on the object receiving it. In simple terms: **different objects can respond differently to the same request.**

Imagine you say “Make a sound!” — a Dog barks, a Cat meows, a Cow moos. The request `makeSound()` is conceptually the same, but the behaviour differs. Likewise, `draw()` draws a circle for Circle and a rectangle for Rectangle.

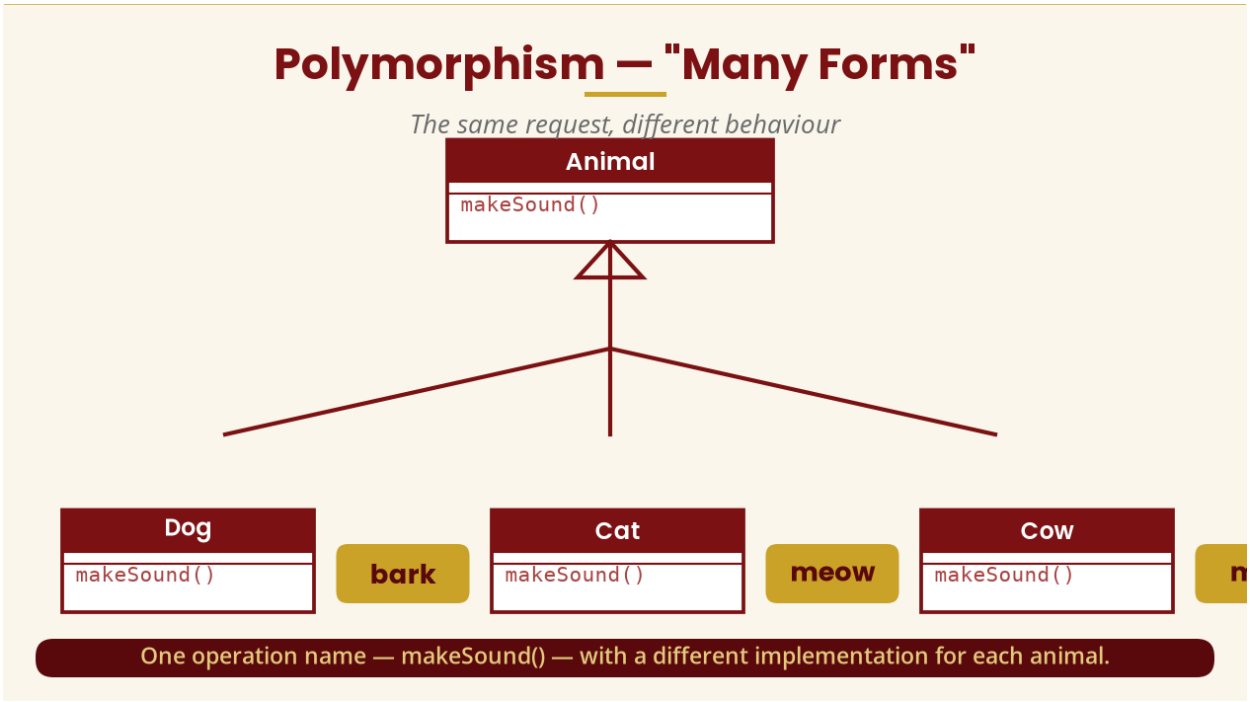

Figure 12 — Polymorphism: one makeSound(), many sounds

13. Why Is Polymorphism Powerful?

Suppose a system must work with different payment types. Payment defines `processPayment()`, and CardPayment, MobileMoney and BankTransfer each implement it differently. The system can issue the general request `processPayment()` without treating every payment type as completely unrelated. A notification system works the same way: EmailNotification, SMSNotification and PushNotification each implement `send()` differently. If we later add WhatsAppNotification, we can add another specialization **without changing every part of the system that simply requests send()** — this is **extensibility**.

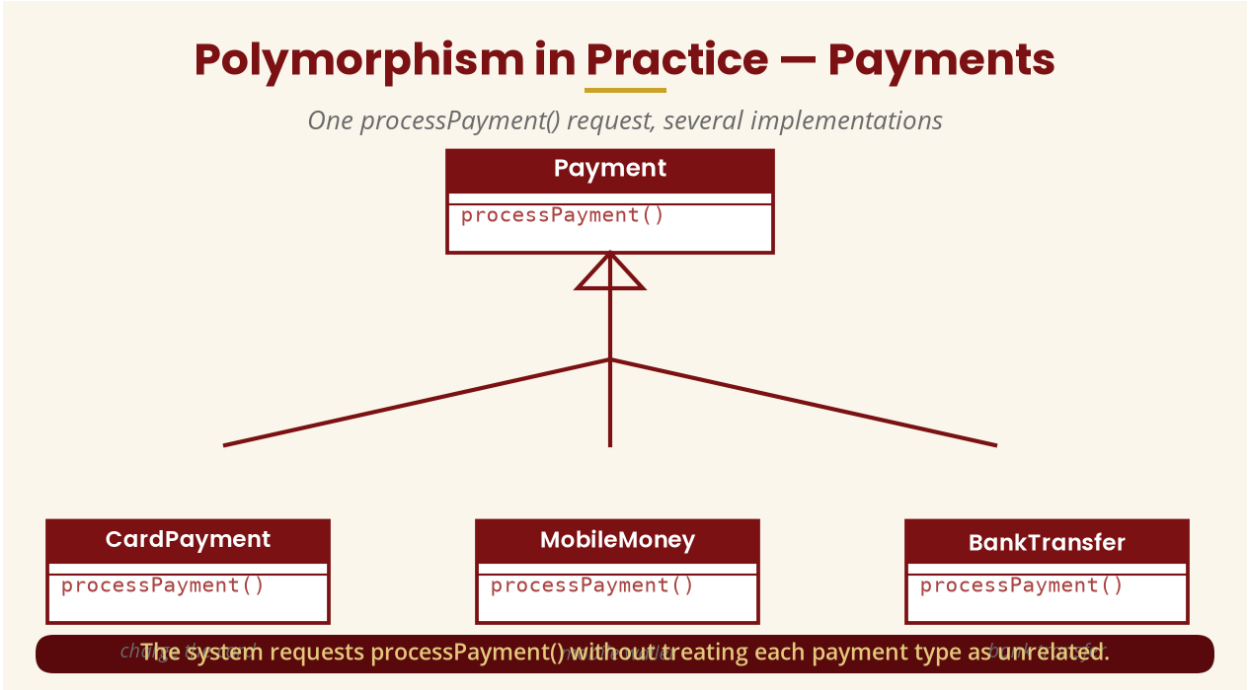

Figure 13 — Polymorphism across payment types

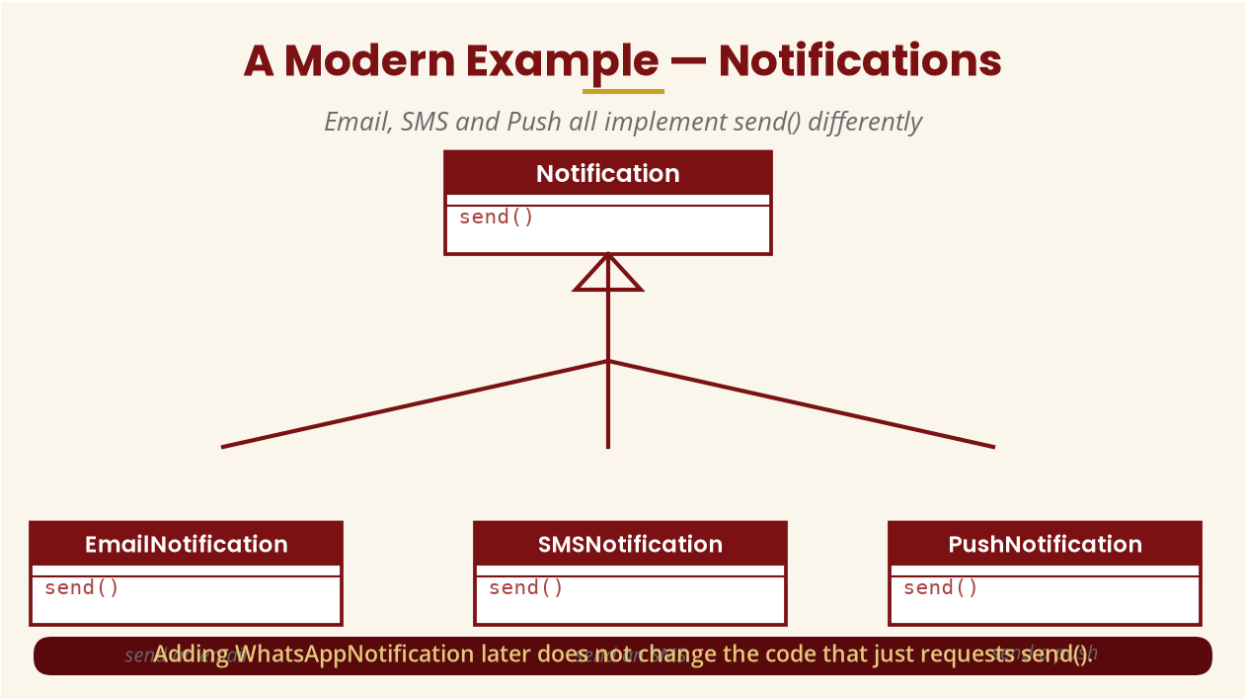

Figure 14 — Polymorphism across notification channels

14. Method Overriding — and the Modelling View

In programming this idea is commonly implemented through **method overriding**: the general class defines `makeSound()`, and each subclass provides a specialized version that overrides the inherited operation. In Java: `Animal a = new Dog(); a.makeSound();` produces "Bark", while `a = new Cat(); a.makeSound();` produces "Meow" — the variable can refer to an Animal, but the actual object's implementation determines the behaviour.

Since this is OOAD, students should first understand the **modelling** idea without programming syntax: we simply model Payment with CardPayment, MobileMoney and BankTransfer all supporting `processPayment()`, each performing it differently.

PART D — GROUPING

15. Grouping — Packages

As systems become larger, diagrams become difficult to understand — an enterprise system with 60 classes in one diagram is confusing. Related elements are therefore organized into meaningful groups: in UML, **packages** provide the mechanism. **Think of a package as a folder.** A University System folder can contain Student Management, Finance, Human Resources, Library and Accommodation; Student Management in turn contains Student, Programme, Course, Registration and Result. Grouping supports organization, readability, modularity, maintenance, team development, navigation and separation of concerns.

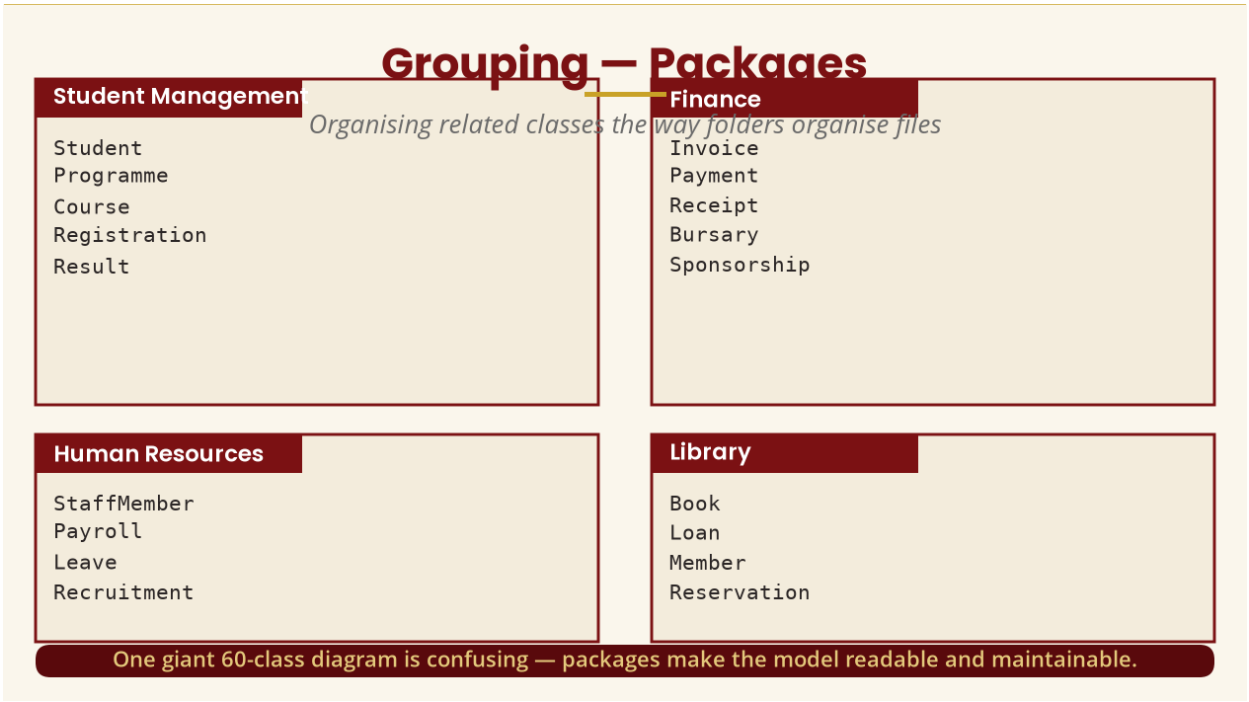

Figure 15 — Grouping classes into packages

16. Complete Example — Online Shopping

All four concepts come together in an online shopping system. **Composition:** Order *-- OrderItem — an OrderItem is strongly associated with a particular Order. **Inheritance:** Payment is generalized into CardPayment, MobileMoney and BankTransfer. **Polymorphism:** all payment types support processPayment() but each performs it differently. **Grouping:** the system is organized into Customer Management, Order Management, Product Management, Payment Management and Delivery Management.

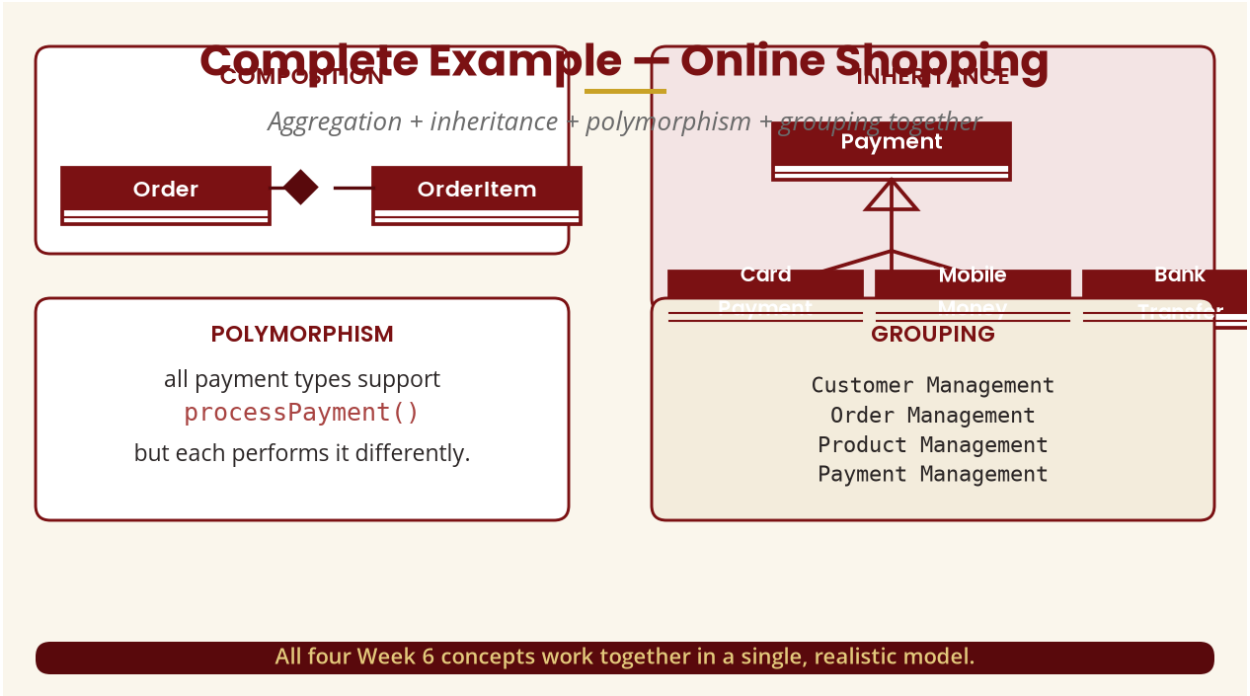

Figure 16 — The four concepts in one model

17. Complete Example — University System

Build a larger model: User --|> Student and User --|> Lecturer (inheritance); `Department` ◇—— `Lecturer` (aggregation); `Student` —— `Registration` —— `Course` (association); `Course` ◆—— `CourseMaterial` (composition, if materials are strongly owned parts). For polymorphism, suppose the university supports different assessment types — Assessment defines `calculateMark()`, specialized into Assignment, Exam, Quiz and Project, each calculating marks differently.

18. Practical Lab and Validation

Using a CASE tool, model aggregation and inheritance: draw `University` ◇—— `Department`, `Order` ◆—— `OrderLine`, and the User -> Student/Lecturer hierarchy, then add a polymorphic operation (e.g. calculateMark()). After modelling, evaluate each decision: is the relationship genuinely whole-part? Is the generalization a genuine is-a? Is polymorphism justified, or would ordinary classes do?

> **EVALUATING A MODEL — KEY QUESTIONS**
>
> Is the aggregation really whole-part, or just an association?
> Are the parts independent of the whole (aggregation) or owned by it (composition)?
> Is every generalization a genuine is-a relationship?
> Does the polymorphic operation have a common meaning across subclasses?
> Is the model organised into readable packages?

19. Week 6 Summary and Key Terms

Term	Simple meaning
Aggregation	Whole-part relationship; hollow diamond; parts can exist independently.
Composition	Strong whole-part; filled diamond; the composite owns its parts.
Inheritance	A specific class acquires features from a general class.
Generalization	The UML relationship drawn as a hollow triangle.
Superclass / Subclass	General class / specialised class.
Polymorphism	One operation name, different behaviour per object.
Method overriding	A subclass provides its own version of an inherited operation.
Extensibility	Adding a new subtype without changing the code that requests the operation.
Grouping / Package	Organising related classes the way folders organise files.

THE BIG PICTURE

Week 5 answered **what objects exist and how they are related**. Week 6 adds the deeper questions: **Is this a whole-part relationship?** (aggregation/composition), **What behaviour can be inherited?** (inheritance), **Can different objects respond differently to the same operation?** (polymorphism), and **How can related classes be organised?** (grouping). Together these make the object model reusable, extensible and understandable — preparing the ground for the dynamic and behavioural modelling of Weeks 7–8.

20. Examination-Style Question

Scenario: An online store has customers, orders, order items and products. Payments can be made by card, mobile money or bank transfer. Each order contains one or more order items. The system is organised into customer, order, product and payment management areas.

Required: (a) draw the classes with a suitable generalization hierarchy for Payment [8]; (b) model Order and OrderItem, explaining whether aggregation or composition is appropriate and why [6]; (c) explain how polymorphism applies to processPayment() and why it supports extensibility [8]; (d) group the classes into packages [4]; (e) justify each modelling decision [4]. **Total: 30 marks.**

21. References and Further Reading

Primary authoritative reference

- Object Management Group. (2017). *Unified Modeling Language (UML), Version 2.5.1.* OMG — the formal UML specification (aggregation as a property of an association end; shared vs composite aggregation; generalization and feature inheritance).
- Object Management Group. *UML* — the UML specification with machine-readable artefacts, including the UML abstract syntax metamodel.

Prescribed and recommended texts

- Mach, E. (2019). *Object Oriented Analysis & Design Cookbook: Introduction to Practical System Modeling.*
- Reddy, V., & Korra, S. (2018). *Object-Oriented Analysis and Design Using UML.*
- The Art of Service. (2021). *Object Oriented Analysis and Design: A Complete Guide.*
- Wixom, B., & Dennis, A. (2018). *Systems Analysis and Design.*
- Blokdyk, G. (2021). *Object-Oriented Analysis and Design Standard Requirements.*
- Wixom, B., & Dennis, A. (2020). *Systems Analysis and Design: An Object-Oriented Approach with UML.*